Stage 2 Implementation Report

1. Project overview and implementation status

Product Finder is a product-discovery and price-comparison application for shoppers. Users can create an account, describe what they want in a chat, save products, and compare offers in the catalog.

Its DeepSeek integration interprets requests, asks follow-up questions, ranks eligible products and explains trade-offs, offers buy-or-wait advice, suggests similar products, and analyzes review trust and summaries.

Current functionality includes login/logout, chat-based search with requirements confirmation, proposed products based on user requirements, on-demand best-price comparisons, buy-now/wait analysis, product list management and notes, product reviews analysis, product summaries, shared-list links, user preferences, and a CLI.

Limitations: the catalog, prices, history, and reviews are synthetic. Chat state is in memory and resets when the backend restarts.

2. Feature-to-implementation mapping

Feature	Stage 1 reference / status	Implementation and collaboration (type)	Key methods	Patterns	Demo video
F01 — Natural-language requirements extraction	Original F01; implemented	The user enters a request in the chat UI. `ChatController` passes it to `AgentService`, which maintains the conversation state and provides the conversation and profile context to `RequirementsExtractor`. The extractor asks the LLM for structured requirements or a follow-up question, validates the requirements, and returns them for the user to review. Hybrid: **AI-based, with deterministic validation and workflow control.**	`handleMessage()`, `extract()`, `confirmRequirements()`	Facade; LLM Adapter, Decorator, Singleton	https://youtu.be/wI7CAK5QsxM?t=1 Alternative case is also shown where the user decides to change their initial requirements from $500 budget to $1000
F02 — Product matching and trade-off explanation	Original F02; implemented	After the user confirms the requirements, `AgentService` calls `ProductMatcher`. The matcher retrieves catalog candidates, validates their attributes, and filters out products that fail the user's constraints or profile settings. It sends only eligible candidates to the LLM, which chooses one and explains trade-offs; the application verifies the choice and returns the product and offer to the chat. **Hybrid: deterministic retrieval and filtering, with AI ranking and explanation.**	`confirmRequirements()`, `match()`, `bestListing()`	Facade; LLM Adapter, Decorator, Singleton	https://youtu.be/wI7CAK5QsxM?t=61
F03 — Price-history buy/wait advice	Original F03; implemented	The user requests advice and supplies urgency. `AgentService` retrieves the selected listing, asks the price-history service for statistics when history exists, and gets the user's budget. `BuyWaitAdvisor` sends that context and urgency to the LLM, then returns its validated buy/wait decision and reason. **Hybrid: deterministic data gathering and validation, with AI advice.**	`recommend()`, `computeStats()`, `advise()`	Facade; LLM Adapter, Decorator, Singleton	https://youtu.be/wI7CAK5QsxM?t=95
F04 — Saved-product list management	Original F04; implemented	The user enters a product name or chooses to save the current recommendation. `SavedList` sends the request to `SavedListController` and `SavedListService`, which resolve the product and manage the list. If the name is approximate, the similar-product suggester can ask the LLM to choose from candidates and the user confirms the suggestion. For a recommendation save, `SavedProductWorkflowService` verifies the current match and stores its offer snapshot. **Hybrid: deterministic list**	`addOrSuggest()`, `confirmAdd()`, `saveRecommendation()`, `addProduct()`, `removeProduct()`	Facade; LLM Adapter, Decorator, Singleton for AI suggestions	https://youtu.be/wI7CAK5QsxM?t=120 Error handling case is shown where the user mistypes 'samsung s25' and the AI is able to infer the correct name

Feature	Stage 1 reference / status	Implementation and collaboration (type)	Key methods	Patterns	Demo video
		operations, with AI-assisted suggestions when needed.			
F05 — Compare retailer prices	Original F05; implemented as an explicit action	The user opens a saved product and selects **Find best price**. `ProductPage` requests a comparison from `ProductPageController`. The controller and workflow check ownership and product status; `ProductService` applies profile preferences through `PriceComparisonService` and returns the lowest eligible current offer, or a no-offer result. **Deterministic.**	`bestPrice()`, `bestListing()`	None	https://youtu.be/wI7CAK5QsxM?t=187
F06 — Share a saved list	Original F06; implemented	The owner requests a share link. `SavedListController` asks `SavedListService` to create or retrieve a token for a non-empty list. A visitor opens the link in `SharedPage`; the backend resolves the token and returns a read-only list with eligible offers. **Deterministic.**	`createShareLink()`, `getSharedList()`	None	https://youtu.be/wI7CAK5QsxM?t=205
F07 — User profile customization	Original F07; implemented	The user edits budget, excluded brands, or allowed conditions in `ProfileEditor`. `ProfileController` passes the update to `ProfileService`, which validates and persists it. The saved preferences are then used by product search and price comparison. **Deterministic.**	`get()`, `update()`, `validate()`	None	https://youtu.be/wI7CAK5QsxM?t=229
F08 — Review-trust analysis	Original F08; implemented	When a product is saved or its page is opened, the workflow requests a trust analysis. `CachedReviewAnalyzer` retrieves and fingerprints review inputs and reuses a matching cached result when available. On a cache miss, `LLMReviewAnalyzer` asks the LLM for a trust label, which is returned to the user and saved item. **Hybrid: deterministic input handling and caching, with AI-generated analysis.**	`analyzeTrust()`, `findByProductIdOrderByReviewedAtDesc()`, and `cacheKey()`	Proxy/cache; LLM Adapter, Decorator, Singleton	https://youtu.be/wI7CAK5QsxM?t=277
F09 — Product and review summary	Original F09; implemented	The user selects **Summarize** on the product page. `ProductPage` sends the request through the product API; the service retrieves the product and eligible reviews. `CachedReviewAnalyzer` reuses a matching summary when possible or delegates to `LLMReviewAnalyzer` to generate one. The summary is returned to the page. **Hybrid: deterministic retrieval and caching, with AI summarization.**	`summarize()`, `reviewInsights()`, `reviewsFor()`	Proxy/cache; LLM Adapter, Decorator, Singleton	https://youtu.be/wI7CAK5QsxM?t=309
F10 — Notes for a saved product	Original F10; implemented	The user enters a note on `ProductPage` and submits it. The saved-list API delegates to `SavedListService`, which checks the saved item, sanitizes unsupported control characters, enforces the length limit, and persists the note. The page displays the saved text and any sanitization notice. **Deterministic.**	`saveNote()`, `note()`	None	https://youtu.be/wI7CAK5QsxM?t=337
CLI Version of App					https://youtu.be/wI7CAK5QsxM?t=361

3. Implemented design patterns

Adapter

- **Problem addressed:** Keep application features independent of DeepSeek's provider-specific HTTP and JSON formats.
- **Participants and roles:** `LLMClient` defines the application's LLM interface. `DeepSeekClient` implements it by translating requests and responses. Feature classes—`ProductMatcher`, `RequirementsExtractor`, `BuyWaitAdvisor`, `SimilarProductSuggester`, and `LLMReviewAnalyzer`—use the interface rather than DeepSeek's API directly.

`SharedLLMClient` sends the HTTP request.

- **Files and methods:** `LLMClient.java`: `complete()`; `DeepSeekClient.java`: `complete()`, `toProviderRequest()`, `fromProviderResponse()`; `SharedLLMClient.java`: `postJson()`.
- **How and why:** The adapter centralizes provider translation, so provider-specific changes are less likely to affect feature logic.
- **Stage 1 comparison:** Matches the planned `LLMClient` / `DeepSeekClient` adapter.

Facade

- **Problem addressed:** Keep API controllers from coordinating many chat subsystems and duplicating state and confirmation rules.
- **Participants and roles:** `ChatController` receives HTTP requests and delegates to `AgentService`. `AgentService` coordinates chat state, profile settings, requirement extraction, product matching, price history, buy/wait advice, and similar-product suggestions. Those collaborators perform the specialized operations.
- **Files and methods:** `ChatController.java`: `postMessage()`, `confirm()`, `buyOrWait()`; `AgentService.java`: `handleMessage()`, `confirmRequirements()`, `retryMatch()`, `recommend()`, `suggestSimilar()`.
- **How and why:** `AgentService` provides a central entry point for the chat workflow and its rules, reducing controller coupling.
- **Stage 1 comparison:** Matches the planned Facade for chat-agent workflows and similar-product suggestions.

Proxy

- **Problem addressed:** Avoid repeated LLM analysis when the relevant product and review content has not changed.
- **Participants and roles:** `ReviewAnalyzer` is the interface used by application services. `CachedReviewAnalyzer` loads current inputs, computes a key, returns a cached result when present, and delegates on a cache miss. `LLMReviewAnalyzer` performs the actual trust or summary analysis using `LLMClient`.
- **Files and methods:** `ReviewAnalyzer.java`: `analyzeTrust()`, `summarize()`; `CachedReviewAnalyzer.java`: `analyzeTrust()`, `summarize()`, `cacheKey()` and `ConcurrentMap.computeIfAbsent()`; `LLMReviewAnalyzer.java`: `analyzeTrust()`, `summarize()`.
- **How and why:** The cache key includes the product, analysis type, and relevant content fingerprint, which helps avoid unnecessary calls and prevents a trust result from being mistaken for a summary.
- **Stage 1 comparison:** The cache-hit/delegate-on-miss intent matches. There are differences: the cache is in memory rather than database-backed, so it is lost on restart; and `LLMReviewAnalyzer` does not implement the same product-ID interface as `ReviewAnalyzer` .

Decorator

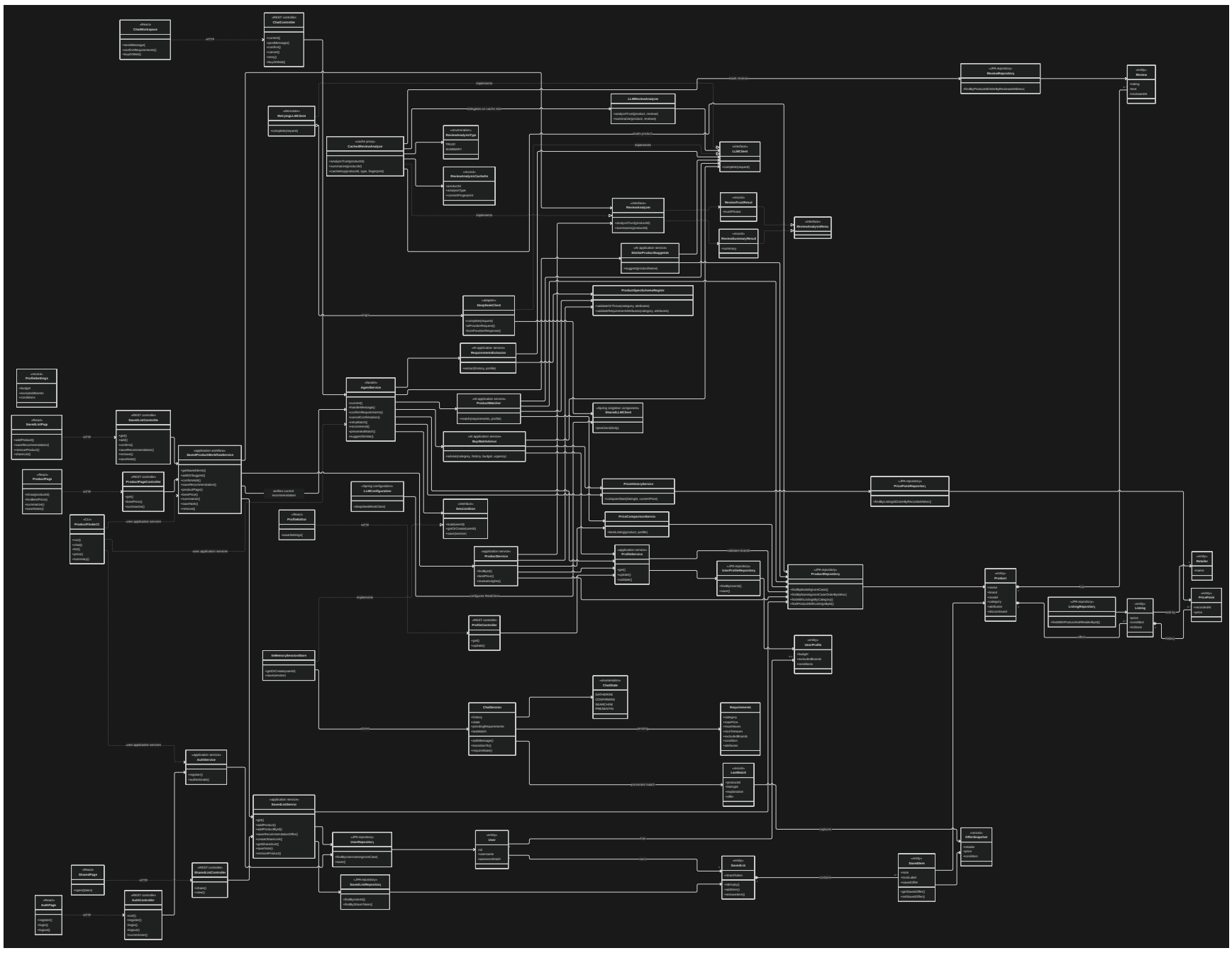
- **Problem addressed:** Add retry and backoff behavior for transient LLM failures without mixing it into provider translation or duplicating it among callers.
- **Participants and roles:** `LLMClient` is the shared interface. `DeepSeekClient` performs the provider-specific call. `RetryingLLMClient` also implements `LLMClient`, wraps another client, and retries transient failures. Spring injects the retrying client as the primary implementation for feature callers.
- **Files and methods:** `RetryingLLMClient.java`: `complete()`, `backoffFor()`, `pause()`; `DeepSeekClient.java`: `complete()`.
- **How and why:** Because the wrapper and underlying client share an interface, retry behavior can be added without changing feature callers or the adapter.
- **Stage 1 comparison:** Matches the planned decorator. The implementation retries only transient failures and bounds attempts and backoff.

Singleton

- **Problem addressed:** Reuse common LLM HTTP configuration and provider settings instead of setting them up independently for each feature.
- **Participants and roles:** `SharedLLMClient` is a Spring-managed component that holds the configured `RestClient` and provider properties. `LLMConfiguration` creates the configured HTTP client. `DeepSeekClient` receives `SharedLLMClient` through dependency injection.
- **Files and methods:** `SharedLLMClient.java`: `postJson()`; `LLMConfiguration.java`: `deepSeekRestClient()`.
- **How and why:** Spring's default singleton bean scope shares managed instances within the application context, centralizing HTTP setup and credentials.
- **Stage 1 comparison:** The reuse goal matches, but the implementation is **not** a manually coded GoF Singleton with `getInstance()`. Stage 1 proposed that global-access approach; the implementation uses constructor injection instead, which keeps dependencies explicit and easier to test. It shares a configured `RestClient`, not a guarantee of one physical network connection.

UML Diagrams

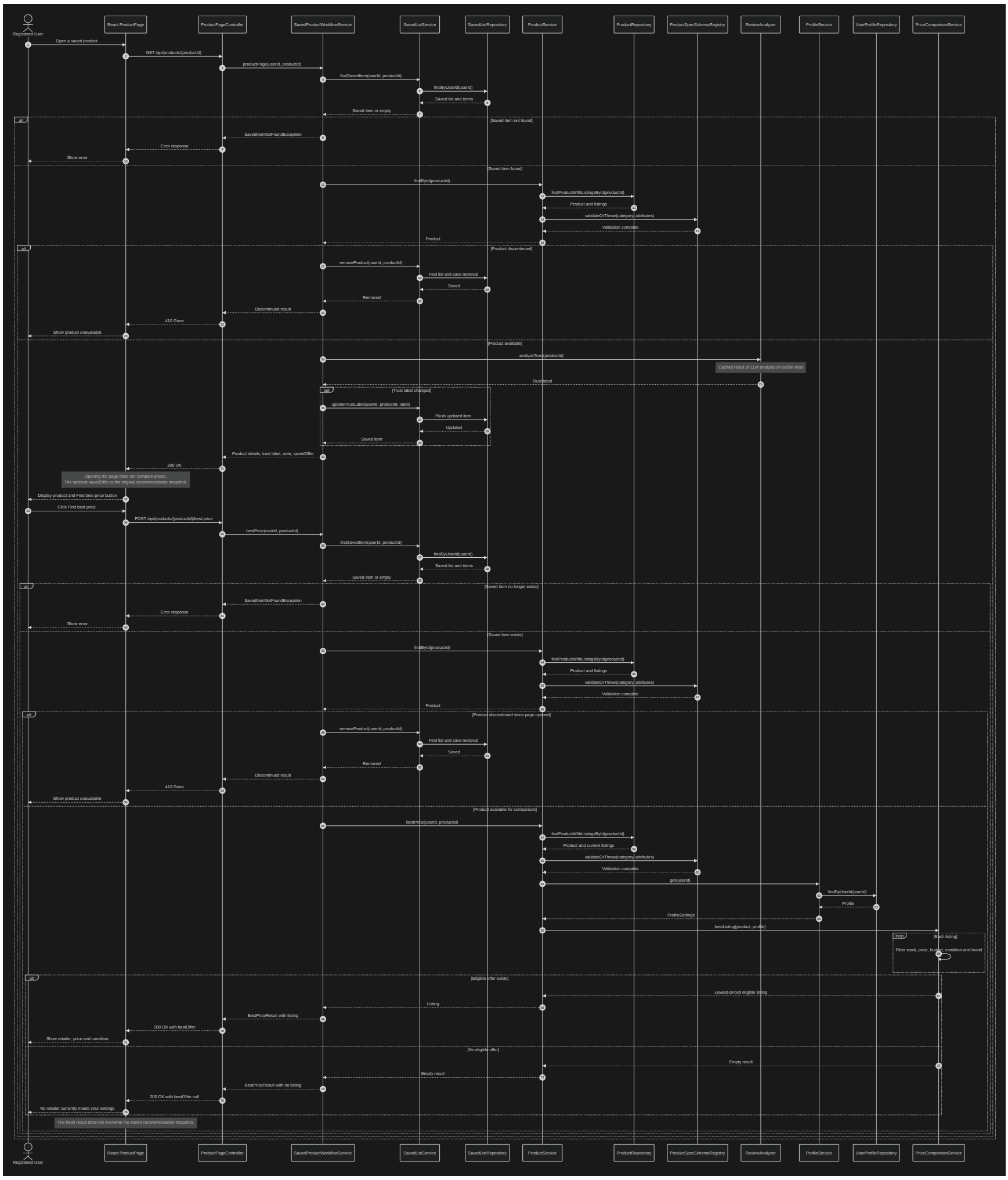
Class Diagram:

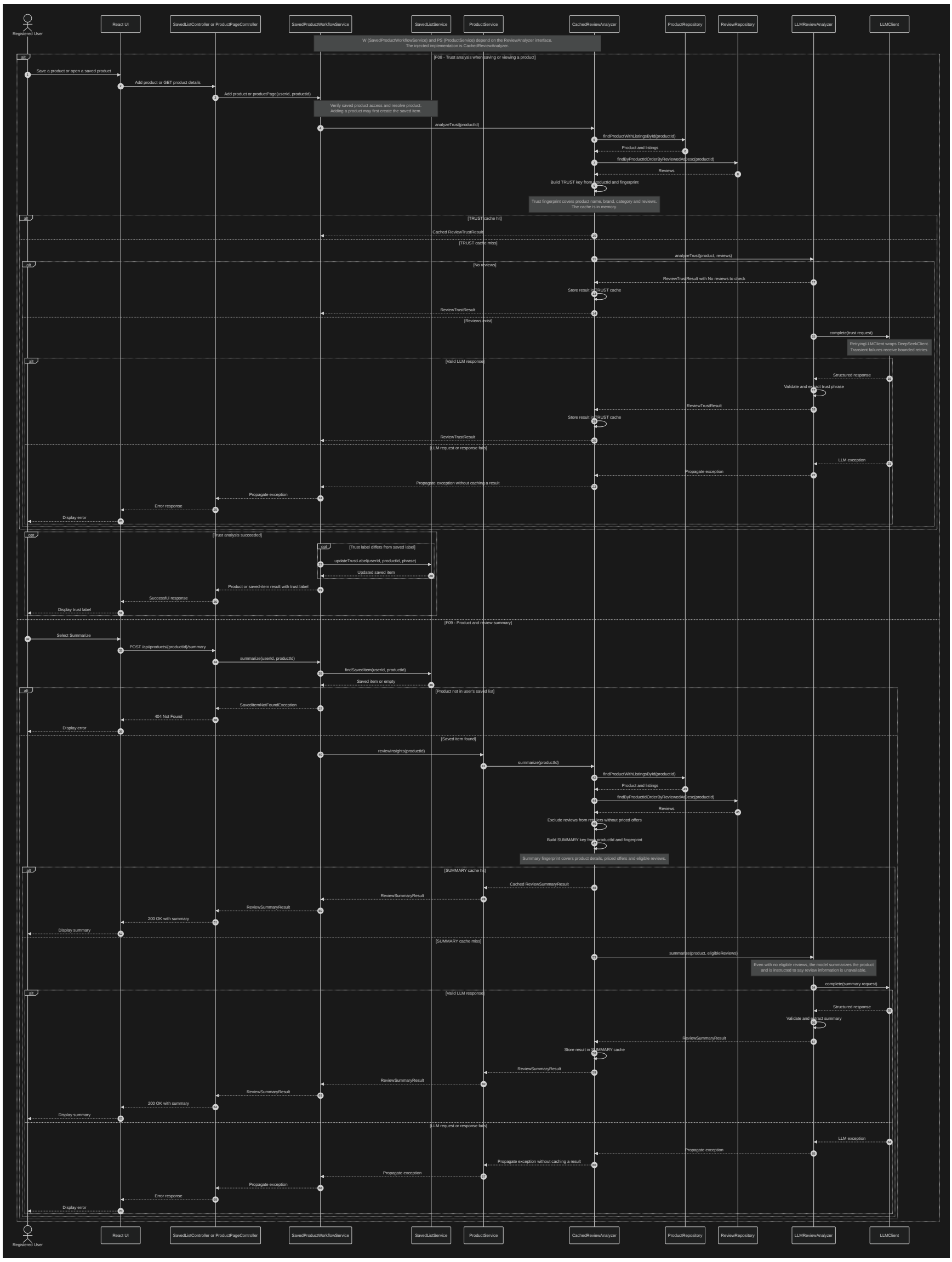


Sequence Diagrams:

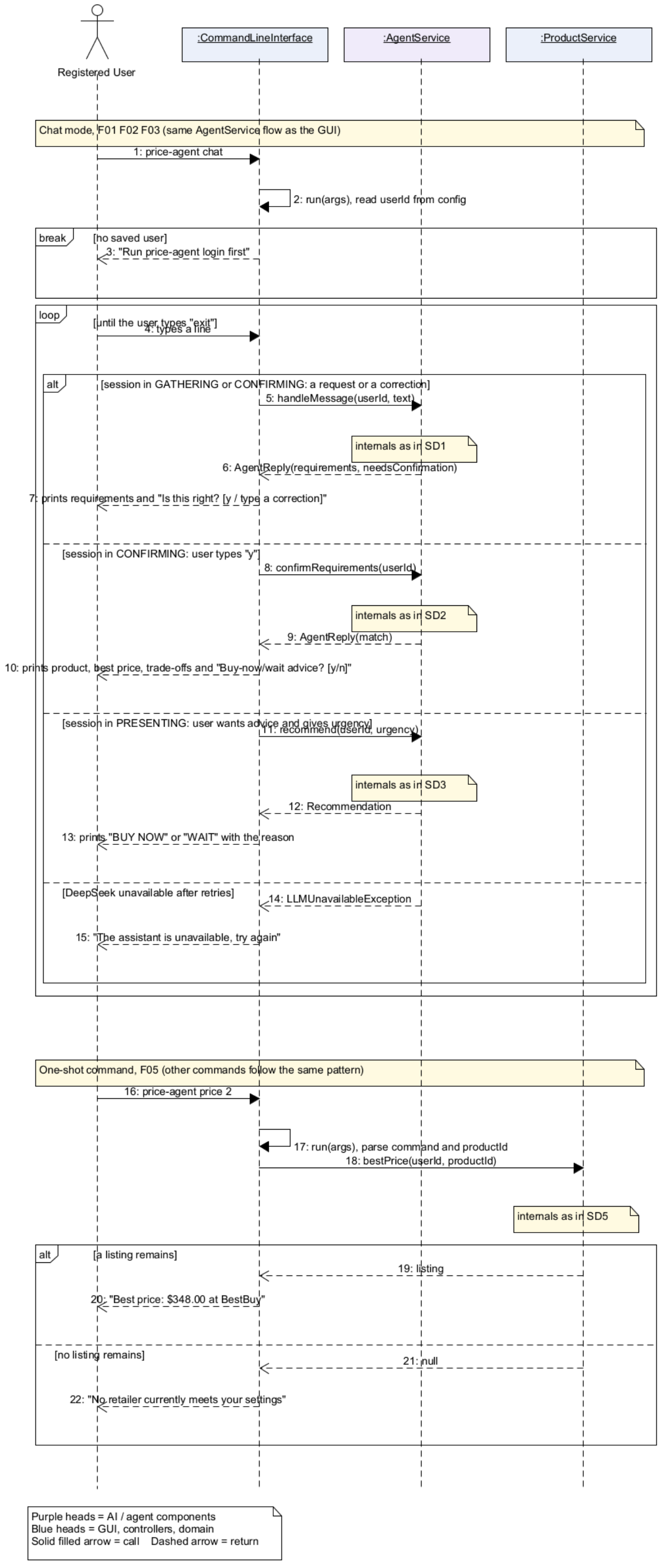
SD5 - View product page and find best price

SD6 - Review summary and trust analysis

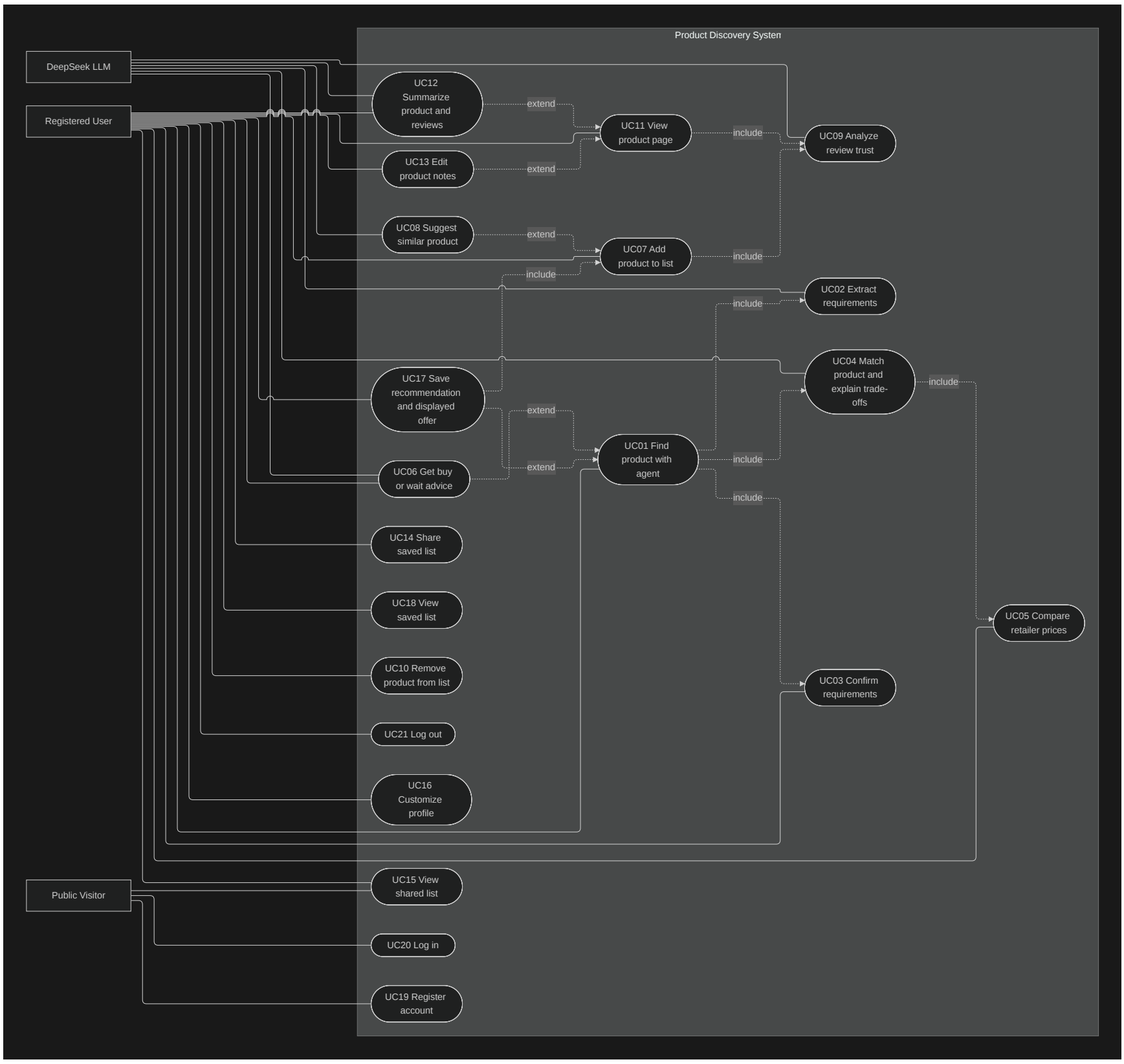




SD10 - Official CLI addition (mostly re-uses GUI’s code foundation)


Registered User
:CommandLineInterface
:AgentService
:ProductService
Chat mode, F01 F02 F03 (same AgentService flow as the GUI)
1: price-agent chat
2: run(args), read userId from config
break
[no saved user]
3: "Run price-agent login first"
loop
[until the user types "exit"]
4: types a line
alt
[session in GATHERING or CONFIRMING: a request or a correction]
5: handleMessage(userId, text)
internals as in SD1
6: AgentReply(requirements, needsConfirmation)
7: prints requirements and "Is this right? [y / type a correction]"
[session in CONFIRMING: user types "y"]
8: confirmRequirements(userId)
internals as in SD2
9: AgentReply(match)
10: prints product, best price, trade-offs and "Buy-now/wait advice? [y/n]"
[session in PRESENTING: user wants advice and gives urgency]
11: recommend(userId, urgency)
internals as in SD3
12: Recommendation
13: prints "BUY NOW" or "WAIT" with the reason
[DeepSeek unavailable after retries]
14: LLMUnavailableException
15: "The assistant is unavailable, try again"
One-shot command, F05 (other commands follow the same pattern)
16: price-agent price 2
17: run(args), parse command and productId
18: bestPrice(userId, productId)
internals as in SD5
alt
[a listing remains]
19: listing
20: "Best price: $348.00 at BestBuy"
[no listing remains]
21: null
22: "No retailer currently meets your settings"
Purple heads = AI / agent components
Blue heads = GUI, controllers, domain
Solid filled arrow = call Dashed arrow = return

Use case diagram:



4. Changes and rationale

Consolidated change record

Features and user-facing behavior

Original design	Implemented change	Reason	Impact
Authentication: Use cases referred to a registered user, but did not define registration, login, logout, or credential storage.	Registration, login, and logout use PostgreSQL-backed accounts and password hashes. Web requests use session authentication; shared-list viewing remains public.	Make the registered-user boundary concrete and protect personal data.	**New supporting trivial feature.** Users authenticate before accessing personal workflows; visitors can still open shared lists.
CLI: A command-line interface was already proposed, including separate `price-agent` commands and a saved user ID.	An interactive `price-agent>` prompt provides login, registration, chat and other commands. Authentication is held in memory for the running CLI; no user-ID config file is saved.	Provide a practical terminal interface using the existing application services.	**Implementation refinement, not a new feature relative to Stage 1.** The interaction differs from the proposed one-shot command model.
F01 — Requirements: Natural-language needs would be converted to structured attributes and confirmed by the user.	Category-specific JSON schemas define and validate product specifications and extracted attributes.	Make requirements consistent across different product types and reject invalid structured data.	The original feature remains; schemas make its previously general structure concrete.
F02 — Matching: The agent would select a best-fit product and explain trade-offs; the exact ranking rules were unspecified.	Java filters eligible products and offers first. DeepSeek selects among eligible candidates and explains trade-offs without a fixed ranking or tie-break formula.	Enforce hard constraints while allowing AI judgment on the best fit.	Refines the original feature rather than replacing it. An ineligible model selection is rejected.
F02/F04 — Save recommendation: UC07 explicitly described adding a product by entering its model name.	The user can also save the current F02 recommendation directly. The saved item records the displayed retailer, price and condition as an offer snapshot.	Preserve the exact product and offer the user saw without requiring another search.	**New save path.** An existing saved item can receive the snapshot too; the displayed offer is identified as potentially outdated.
F04 — Add a product: Enter a specific model; an incorrect model could produce a suggested alternative for confirmation.	Exact product-name matching is used. For approximate names, a deterministic token match can produce a suggestion; otherwise	Support approximate input while avoiding unnecessary AI calls where a strong match exists.	Retains the original need and confirmation interaction, with a more specific matching strategy.

Original design	Implemented change	Reason	Impact
	DeepSeek selects from similar candidates. The user confirms before adding.		
F05 — Compare prices: The user clicks **Find best price** on the saved product page; comparison is also part of a recommendation.	The saved page displays a recommendation snapshot if one exists. A fresh comparison is performed when the user clicks **Find best price**; it does not overwrite the snapshot.	Distinguish the previously displayed offer from a newly calculated price.	**The button is not new relative to Stage 1.** The snapshot and its separation from a fresh comparison are new.
F06 — Share list: Generate a public page, open it and copy its link. Empty lists cannot be shared.	The UI creates and displays a share link and attempts to copy it; if clipboard access fails, the user can copy it manually. It does not automatically open the page.	Make the link available even when clipboard access is unavailable.	Public read-only sharing remains; the post-share navigation differs from Stage 1.
F08 — Review trust: Run review detection when adding a product and display a short trust phrase.	Trust analysis also runs when opening the saved product page. It can reuse a cached result and updates the saved label when it changes. The model receives guidance favouring a positive label for established brands in ordinary cases.	Refresh the displayed label when relevant inputs change and reduce repeated LLM calls.	The feature remains, with an added refresh point.l
F09 — Summary: Summarize product information and reviews on request. The original analyzer had one generic analysis operation.	Summary and trust are separate operations and result types. `reviewInsights()` returns the summary, not the trust label.	Prevent the two different outputs from being confused or colliding in the cache.	Users still request a summary separately; the internal contract is more precise.
F10 — Notes: Users could read and save notes; problematic characters were identified as an error case.	Notes have a length limit and disallowed control characters are removed before saving.	Handle the identified input problem consistently.	The original feature remains with defined input handling.

Architecture and classes

Original design	Implemented change	Reason	Impact
The technology choices and local authentication/database details were not fully specified.	Java/Spring Boot backend, React frontend, local PostgreSQL, migrations and synthetic catalog data.	Resolve the implementation choices needed to build the designed system.	The original architecture is made concrete; there is no deployment target.
`CommandLineInterface` called services directly; the web interface used controllers.	`ProductFinderCli` is an interactive Spring Boot runner that calls application services directly.	Reuse backend behavior without routing terminal commands through HTTP.	Web and CLI share much of the application layer, but their interactions and some checks differ.
`ProductController` delegated to an already-designed `ProductService`; the Stage 1 sequence included discontinued-product handling.	`ProductPageController` delegates saved-product requests to `SavedProductWorkflowService`, which coordinates ownership, discontinued items, trust refresh and offer snapshots.	Centralize saved-product workflow rules outside HTTP handling.	**New workflow class**, not a new `ProductService`. Web product-page and price requests get the saved-item checks.
`ProductService` handled product-related operations.	`ProductService` handles lookup, profile-aware price comparison and summary delegation; it does not itself remove discontinued saved items.	Separate general product operations from saved-item workflow rules.	Clarifies class responsibilities. The CLI `product` command uses the workflow, but its `price` command calls the product service directly and bypasses the web workflow's saved-item and discontinued checks.
`SavedListController` was depicted handling list, sharing and profile operations.	Responsibilities are split among `SavedListController`, `SharedListController` and `ProfileController`.	Separate distinct API responsibilities and public sharing from personal operations.	Controller organization changes; the underlying list, sharing and settings needs remain.
`SessionStore` and chat states were designed, but the storage implementation and full state representation were not explicit.	`InMemorySessionStore` stores chat state in the backend process; the current match is retained for follow-up actions and recommendation saving.	Enforce the chat workflow and verify that a saved recommendation belongs to the current session.	Chat state resets when the backend restarts.
`SavedItem` contained a note and trust label; no saved offer was shown.	Saved items can also hold the recommendation offer snapshot.	Retain what was shown when the recommendation was made.	Product pages can show that historical snapshot without immediately recalculating it.
One `ReviewAnalyzer.analyze(productId)` operation was shown for review analysis.	`ReviewAnalyzer` exposes separate trust and summary operations with typed results and analysis-specific cache keys.	Separate two outputs with different inputs and purposes.	Trust and summary cannot be mistaken for one another in the cache.

Design patterns

Original design	Implemented change	Reason	Impact
Facade: `AgentService` was the entry point for agent workflows, principally used by the chat controller.	`AgentService` also serves the CLI and supports recommendation verification for the saved-product workflow.	Keep chat state and agent rules below interface code.	Facade is **retained**, not replaced; it has additional clients.
Proxy: Both the cached analyzer and real analyzer were depicted as implementing the same `ReviewAnalyzer` interface.	`CachedReviewAnalyzer` implements the interface and delegates cache misses to `LLMReviewAnalyzer`, which does **not** implement that same interface.	Centralize loading, fingerprinting and caching while keeping LLM analysis separate.	The implementation has **caching-proxy behavior**, but is not a fully transparent classic proxy with interchangeable proxy and real subject. The cache is in memory.
Adapter: `DeepSeekClient` adapted the application's LLM interface to DeepSeek.	`DeepSeekClient` implements the application-owned LLM contract and maps provider requests and responses.	Isolate provider-specific protocol details.	Adapter is **retained**, not replaced.
Decorator: `RetryingLLMClient` wrapped the LLM client to retry temporary failures.	`RetryingLLMClient` wraps the client with bounded retry behavior.	Keep retry policy separate from provider translation.	Decorator is **retained**, not replaced.
Singleton: `SharedLLMClient` was shown with a static `getInstance()` global access point.	`SharedLLMClient` is provided as a Spring-managed singleton-scoped component.	Share the client through dependency injection rather than a static global accessor.	The shared-instance goal remains, but the **classic static Singleton implementation is replaced** by Spring lifecycle management —one instance per application context.

Section 3 Notes (Changes from Stage 1)

1. CLI implementation and refinements

Implemented feature and behavior

The application provides an interactive terminal interface using `ProductFinderCli`. After starting the application in CLI mode, the user sees the `price-agent>` prompt and can register or log in.

The `chat` command starts a nested chat session. The user describes a product, reviews the extracted requirements, and uses `/confirm` to approve the search. They can also use `/cancel`, `/retry`, `/buywait`, and `/back`. The agent returns a product recommendation, retailer price, and trade-off explanation.

Other commands include `list`, `add`, `confirm-add`, `remove`, `product`, `price`, `summary`, `note`, `profile`, `settings`, and `share`. `clear` or `cls` clears the terminal, while `exit` leaves the current prompt or ends the CLI.

Authentication is held in the running CLI's `currentUser` field; **the CLI does not save the user ID to a configuration file**. Errors are printed to the terminal.

What changed and why

Stage 1 proposed a `CommandLineInterface`; the implemented class is `ProductFinderCli`. It is an interactive Spring Boot command-line runner, rather than a set of independently invoked `price-agent` operating-system commands.

It calls application services directly in the same process, without going through REST controllers. This reuses the agent and saved-product workflows while allowing the terminal and web interfaces to present them differently.

Original need and scope

The CLI addresses the same product-discovery and saved-list needs through a terminal interface. It was **already present in the Stage 1 design**, so its implementation is not an entirely new feature relative to that document.

Most registered-user workflows are available from the terminal.

Affected use cases, classes, methods, and patterns

- **Use cases:** The existing registered-user use cases are served through an additional interface; no new use case is required solely for the CLI.
- **Classes:** `ProductFinderCli` calls `AuthService`, `AgentService`, `SavedProductWorkflowService`, `ProductService`, `ProfileService`, and `SavedListService`.
- **Key methods:** `ProductFinderCli.run()` and its command handlers; `AgentService.handleMessage()`, `confirmRequirements()`, and `recommend()`; `SavedProductWorkflowService.addOrSuggest()`, `productPage()`, `summarize()`, and `saveNote()`.
- **Patterns:** The `AgentService` Facade serves both `ChatController` and the CLI. AI-dependent workflows also reuse the existing LLM adapter, retry decorator, shared client, and review-analysis cache where applicable.

2. Modified F04, F05, F08, and F09 saved-product workflows

Implemented feature and behavior

The shared saved-product workflow is implemented by `SavedProductWorkflowService`:

- **F04 — Saved products:** Name-based additions use an exact product name or offer a similar-product suggestion requiring confirmation. Saving an F02 recommendation records the exact product and its displayed retailer, price, and condition as an offer snapshot.
- **F05 — Price comparison:** Opening a saved product page does **not** automatically compare prices. A stored recommendation snapshot is displayed if available; the user selects **Find best price** for a fresh comparison. The fresh result does not overwrite the stored snapshot.
- **F08 — Trust label:** Adding or opening a saved product refreshes its trust label through `ReviewAnalyzer`.
- **F09 — Summary:** The user requests a separate product-and-review summary. `ProductService.reviewInsights()` returns a summary; it does **not** return the trust label.

For web product-page and best-price requests, the workflow checks ownership and removes a discontinued product from the saved list. The CLI `product` command uses this product-page workflow, but the CLI `price` command bypasses those workflow checks, as noted above.

What changed and why

Stage 1 already described `ProductService` and a delegating `ProductController`. The current implementation instead uses `ProductPageController` → `SavedProductWorkflowService` → underlying services. This adds an explicit application workflow layer for ownership checks, discontinued-product handling, review-label refresh, and saved-offer handling.

`ProductService` remains responsible for product lookup, profile-aware price comparison, and delegating summary requests. It does **not** itself remove discontinued products or depend on `SavedListService` for that removal.

These changes keep HTTP handling separate from application workflow rules and allow the CLI to reuse several of the same services. The explicit price button and recommendation snapshot are user-visible behavior changes, not merely internal refactoring.

Original need and scope

The original needs—viewing a saved product, comparing retailer prices, checking review trust, and reading a summary—remain. The scope was refined to distinguish:

1. The offer shown in an F02 recommendation and saved as a potentially outdated snapshot.
2. A fresh F05 comparison requested by the user.
3. An F08 trust label and a distinct F09 summary.

Affected use cases, classes, methods, and patterns

- **Use cases:** UC05 Compare retailer prices; UC07 Add product; UC09 Detect fake reviews; UC11 View product page; UC12 Summarize product and reviews.
- **Classes:** `ProductPageController` handles HTTP; `SavedProductWorkflowService` coordinates saved-product rules; `SavedListService` manages saved items; `ProductService` handles product lookup, price comparison, and summary delegation; `ReviewAnalyzer` supplies typed trust and summary results.
- **Key methods:** `SavedProductWorkflowService.saveRecommendation()`, `productPage()`, `bestPrice()`, and `summarize()`; `SavedListService.saveRecommendationOffer()`; `ProductService.findById()`, `bestPrice()`, and `reviewInsights()`; `ReviewAnalyzer.analyzeTrust()` and `summarize()`.
- **Patterns:** Review analysis uses caching-proxy behavior. The LLM paths use the existing Adapter and Decorator; these changes do not introduce a new design pattern.

5. AI tools and collaboration log

Tool or platform	LLM/model	Features, classes or development tasks	Token usage	Money cost
GitHub Copilot in VS Code	GPT-6 Luna and GPT-6.1 Terra, auto selected according to task complexity	Assisted with implementation, testing, debugging and documentation across the Java backend and React frontend. Work	Exact token total unavailable. Copilot reports **269.4 of 1,500 credits used**	Used Copilot Free initially, then purchased a **$10 Copilot Pro subscription**.

Tool or platform	LLM/model	Features, classes or development tasks	Token usage	Money cost
		included the agent workflow, authentication, saved-product workflows, recommendation snapshots, review analysis, and diagrams.		
DeepSeek API	DeepSeek Flash	Integrated into the application for requirements extraction, product matching and trade-offs, buy/wait advice, similar-product suggestions, review trust analysis, and summaries. Related classes include `DeepSeekClient` and `LLMReviewAnalyzer`.	**1,186,444 tokens**, as shown in the DeepSeek platform portal.	**$0.17**

6. Development and verification summary

Milestone 1 bootstrapped the project and domain, establishing a Spring Boot backend, a React frontend, database migrations, core entities and value objects, repository interfaces, synthetic seed data, and a chat flow with explicit states (gathering, confirming, searching, presenting). Milestone 2 added identity and persistence, with PostgreSQL-backed users, secure password hashing, registration, login, and logout, authorization for registered-user operations, local configuration, and repository integration tests against PostgreSQL. Milestones 3 and 4 implemented the product schemas and deterministic services and added the DeepSeek adapter. Milestones 5 through 7 built agent-driven search and saved-product workflows, and implemented review trust and summaries. Milestones 8 through 10 delivered the sharing API, connected the React UI and CLI, and verified end-to-end behavior through testing with live DeepSeek requests. Subsequent work has focused on polish and bug fixes: category-specific product details are now displayed properly instead of as raw JSON, text formatting has been cleaned up, unnecessary UI indicators have been removed, additional datasets have been added to the database, and the chatbox supports Enter-key submission. The previously missing “find best price” button in F05 is now implemented, and a bug that caused the price shown in the chatbox to differ from the one in F05 has been resolved.

- Commits involved:
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/6e914a9d20c4602e79e05ecee7dee1a740ccb961
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/05532be544da760f7b6f7874e058dbe91855f6f5
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/daec6b5404fa6aeff73d1d985e56b5aebe0d4d8d
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/481c0746c3b53238b042fa6622b5641c621e6932
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/6d4f72de1a35e5989e084650c207e496c9a4c1fb
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/70073a0467708df8b1aa7f5a6cb64ddeb9f7af3a
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/d75f34ad3ccc9b16fe9d798779dda8a13f823dce
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/6de8b6da2ff8e65732165d7ef2f1f833c8d35f03
  - https://github.com/jordantran092/AI-Agent-App-Product-Price-Comparison/commit/6168deb39d400956e6d3d7233acd137f1a0b438f

- Checks performed to verify implementation
  - **Backend:** 94 automated tests passed. They covered authentication, chat, product matching, saved lists, pricing, sharing, reviews, and DeepSeek error handling.
  - **Frontend:** 16 tests passed, along with the TypeScript/Vite build and ESLint.
  - **Database:** PostgreSQL startup, migrations, repositories and API workflows were checked.
  - **Manual checks:** Browser and CLI smoke tests were performed. Live DeepSeek calls successfully exercised requirements extraction and product ranking.

- Important defects corrected
  - **Saved products:** Recommendation saves could lose the displayed offer or select the wrong variant; the exact product and offer are now saved.
  - **Price comparison:** Opening a saved product triggered an unwanted comparison; comparison now happens when the user clicks **Find best price**.
  - **Sharing:** Links pointed to the wrong host, and a database fetch caused shared-list retrieval to fail.
  - **Database:** A Flyway checksum mismatch blocked startup; migration history and the saved-offer schema were repaired.
  - **Product lookup and AI errors:** Some existing products caused unnecessary DeepSeek calls; matching and incomplete-response diagnostics were improved.
  - **Chat and display:** Fixed an invalid retry option, unreadable requirements, and duplicate follow-up text.
  - **CLI:** Fixed screen clearing on Windows.

- Remaining limitations
  - The catalog, prices, history, and reviews are synthetic.
  - Chat state is in memory and resets when the backend restarts.
  - The CLI does not have a direct command to save the current recommendation with its offer snapshot.

7. Demonstration index

Refer to: Feature-to-implementation mapping